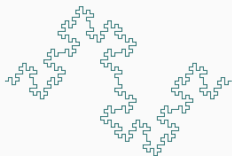


Coordination et Consensus Multiagents (1)

N. Maudet

2024–2025



CoCoMA — Master ANDROIDE

Auctions are a very convenient way to allocate resources or tasks:

- agents submit **bids** on single or bundle of items, ie. prices that are ready to pay for that item/bundle.
- an **auctioneer** gathers the bids and computes the optimal allocation
- agents pay according to the payment scheme.

Summary of the course

Single Item Auctions

Matching by Auctioning Items

Combinatorial Auctions

Application: Auctions for Robot Coordination

Single Item Auctions

Basic Setting: Allocating a Single Item

There are many different types of auctions: English, Dutch, etc.

- ascending vs. descending
- sealed bids vs. open bids
- first price vs. second price

Truthfulness: Allocating an Item

Suppose we want to allocate a single item among n agents and implement the following rule:

- agents report sealed bids
- the item goes to the agent who values it the most
- the price paid by this agent is the **second highest price**

Truthfulness: Allocating an Item

Suppose we want to allocate a single item among n agents and implement the following rule:

- agents report sealed bids
- the item goes to the agent who values it the most
- the price paid by this agent is the **second highest price**

This is called a **Vickrey auction**. It has the enjoyable property that it is a **dominant strategy** for agents to reveal their true preferences ($\bar{v}(o)$). It is said to be **strategy-proof**.

Truthfulness: Allocating an Item

Intuition:

- suppose the agent i overbids: but the only case where this affects the allocation is if an agent bidded between $v_i(o)$ and $\bar{v}_i(o)$: i would get the item for a price higher than valuation!
- suppose that agent i underbids: only makes less likely to get the item...

Matching by Auctioning Items

Matching by Auctioning Items

- assume cardinal preferences of agents
- items not initially held by agents
- interestingly, a simple protocol based on auctions allows to find a matching

D. Bertsekas. *Auction Algorithms for Network Flow Problems*. Computational Optimization and Applications, 1992.

Matching by Auctioning Items

1. assign to each item o a price p_o initialized to 0;
2. pick an agent i to whom no item is assigned yet
3. select the item o^* such that, at its current price p_{o^*} and for agent i , offers the highest utility

$$o^* \in \operatorname{argmax}_{o \in \mathcal{R}} (v_i(o) - p_o)$$

4. agent i then bids by augmenting the price as much as possible in the limit that another object doesn't become more attractive
5. assign i receives item o (which is un-assigned to the agent currently holding it, if any)

Matching by Auctioning Items

Suppose agents have the following utilities:

	a	b	c
x	1	0	4
y	6	8	0
z	0	6	1

turn	p_a	p_b	p_c	agent	object	bid	allocation
0	0	0	0	x	c	3	$\langle x, c \rangle$

Matching by Auctioning Items

Suppose agents have the following utilities:

	a	b	c
x	1	0	4
y	6	8	0
z	0	6	1

turn	p_a	p_b	p_c	agent	object	bid	allocation
0	0	0	0	x	c	3	$\langle x, c \rangle$
1	0	0	3	y	b	2	$\langle x, c \rangle, \langle y, b \rangle$

Matching by Auctioning Items

Suppose agents have the following utilities:

	a	b	c
x	1	0	4
y	6	8	0
z	0	6	1

turn	p_a	p_b	p_c	agent	object	bid	allocation
0	0	0	0	x	c	3	$\langle x, c \rangle$
1	0	0	3	y	b	2	$\langle x, c \rangle, \langle y, b \rangle$
2	0	2	3	z	b	4	$\langle x, c \rangle, \langle z, b \rangle$

Matching by Auctioning Items

Suppose agents have the following utilities:

	a	b	c
x	1	0	4
y	6	8	0
z	0	6	1

turn	p_a	p_b	p_c	agent	object	bid	allocation
0	0	0	0	x	c	3	$\langle x, c \rangle$
1	0	0	3	y	b	2	$\langle x, c \rangle, \langle y, b \rangle$
2	0	2	3	z	b	4	$\langle x, c \rangle, \langle z, b \rangle$
3	0	6	3	y	a	4	$\langle x, c \rangle, \langle z, b \rangle, \langle y, a \rangle$

The auction stops with allocation

$A(x) = \{c\}$, $A(z) = \{b\}$, $A(y) = \{a\}$ and price vector $\langle 4, 6, 3 \rangle$.

The utilities of agents are respectively: 1 for x, 2 for y, et 0 for z.

Matching by Auctioning Items: Properties

- An allocation A and a price vector are in competitive equilibrium (CE) when, for any agent i , it holds that

$$\forall o, u_i(A(i)) \geq u_i(o)$$

- the auction terminates (although there is subtlety) in a CE
- more specifically:
 - (1) if an allocation with price vector are CE then this allocation is optimal (in the sense of maximizing sum of utilities here), and
 - (2) for any optimal allocation A , there exists a price vector $\vec{p}$ such that A with $\vec{p}$ is CE.

Combinatorial Auctions

Combinatorial Auctions

In the combinatorial auction setting, agents bid on **bundle** of items (as opposed to single items) There are two questions which occur:

- how can agents express preferences on bundles of items?
- how can the auctioneer compute the optimal (= maximizing the revenue) allocation given the submitted bids

This is known as the Winner Determination Problem (of course, NP-complete)

Cramton, Shoham, Steinberg. *Combinatorial Auctions*. CUP, 2006.

Combinatorial Auctions

This occurs when items have synergies between them, either:

- **complementarities**: super-additive valuations
- **substitutabilities**: sub-additive valuations

Note: we will assume that valuations are **monotonic**, that is

$$v(B') \geq v(B) \text{ when } B' \supseteq B$$

(related to an hypothesis of **free disposal** : if some items are not valuable we can simply drop them freely)

Combinatorial Domains: Example from Robot Coordination

A robot has to visit several sites.

Let us assume utility is 10 per site visited - cost of tour

- Suppose the robot has to come back to its base after visiting each site

Combinatorial Domains: Example from Robot Coordination

A robot has to visit several sites.

Let us assume utility is 10 per site visited - cost of tour

- Suppose the robot has to come back to its base after visiting each site → additive domain

Combinatorial Domains: Example from Robot Coordination

A robot has to visit several sites.

Let us assume utility is 10 per site visited - cost of tour

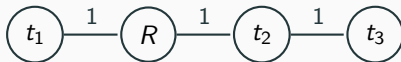
- Suppose the robot has to come back to its base after visiting each site → additive domain
- Suppose the robot does not have to come back to its base (not even at the end of the tour)

Combinatorial Domains: Example from Robot Coordination

A robot has to visit several sites.

Let us assume utility is 10 per site visited - cost of tour

- Suppose the robot has to come back to its base after visiting each site → **additive** domain
- Suppose the robot does not have to come back to its base (not even at the end of the tour) → can be **subadditive** or **superadditive**



Picking t_2 and t_3 : superadditive ($v(t_2) = 9, v(t_3) = 8, v(t_2, t_3) = 18$)
Picking t_1 and t_2 : subadditive ($v(t_1) = 9, v(t_2) = 9, v(t_1, t_2) = 17$)

Bidding Languages (1)

Consider **single-minded** (aka **atomic**) bids.

Agents submit a single bid on one bundle: $\langle B, p \rangle$

The agent is committed to pay p for any $S \supseteq B$

Problem: not expressive at all.

N. Nisan. *Bidding Languages*. Chapter 9 in *Combinatorial Auctions*, 2006.

Bidding Languages: XOR language

When an agent submits bids of the form:

$$\langle B_1, p_1 \rangle \text{XOR} \langle B_2, p_2 \rangle \dots \langle B_n, p_n \rangle$$

it is committed to pay, for a set of items S , $\max_{i|S_i \subseteq S} p_i$

Intuition: “At most one of the bids”

Problem: not compact

N. Nisan. *Bidding Languages*. Chapter 9 in *Combinatorial Auctions*, 2006.

Bidding Languages: OR language

When an agent submits bids of the form:

$$\langle B_1, p_1 \rangle \text{OR} \langle B_2, p_2 \rangle \dots \langle B_n, p_n \rangle$$

it is committed to pay, for a set of items S , the maximum (over all the non-overlapping collections W of subsets S_i) of $\sum_{i \in W} p_i$.

Intuition: same as different atomic bids from several bidders

Problem: cannot represent sub-additive valuations

N. Nisan. *Bidding Languages*. Chapter 9 in *Combinatorial Auctions*, 2006.

Bidding Languages: OR* language

Idea: introduce fake items to discard some combinations of the OR language. For example (f being a fake item):

$$\langle \{A, f\}, 5 \rangle \text{OR} \langle \{B, f\}, 5 \rangle \text{OR} \langle \{A, B\}, 8 \rangle$$

allows to make $\{A, f\}$ and $\{B, f\}$ mutually exclusive, hence to capture sub-additive valuations.

Combinatorial Auctions: example

Solve the following combinatorial auction:

- agent 1: $\langle \{a\}, 2 \rangle$ OR $\langle \{b, c\}, 7 \rangle$
- agent 2: $\langle \{a, b\}, 5 \rangle$ OR $\langle \{c\}, 2 \rangle$
- agent 3: $\langle \{a\}, 3 \rangle$ OR $\langle \{b\}, 3 \rangle$ OR $\langle \{c\}, 3 \rangle$

Application: Auctions for Robot Coordination

Many applications require both:

- task-assignment (where to go)
- multi-agent path-finding (how to go)

Some approaches combine both.

Multi-robot Task-Assignment Problems

Gerkey and Mataric propose the first taxonomy of multi-robot task-assignment problems:

- each robot is capable of executing as most one task at a time (ST) vs. multiple tasks at a time (MT)
- each task requires exactly one robot (SR) vs. multiple robots (MR) to achieve it
- instantaneous allocation (IA) vs. time extended allocation (TA) of tasks to robots

however an important assumption is that of task independence.

Gerkey, Mataric. *A formal analysis and taxonomy of task allocation in multi-robot systems*. IJRR-2004.

Multi-robot Exploration Problems

Several nodes (m) to be visited (only once) by different agents (n)
[note: there may be more nodes than agents]

Which objective?: allocate nodes to agents and find a path for each agent such ...

- (MiniSum) the sum of all the paths is minimized

Multi-robot Exploration Problems

Several nodes (m) to be visited (only once) by different agents (n)
[note: there may be more nodes than agents]

Which objective?: allocate nodes to agents and find a path for each agent such ...

- (MiniSum) the sum of all the paths is minimized
- (MiniMax) the max path cost over the robots is minimized

Multi-robot Exploration Problems

Several nodes (m) to be visited (only once) by different agents (n)
[note: there may be more nodes than agents]

Which objective?: allocate nodes to agents and find a path for each agent such ...

- (MiniSum) the sum of all the paths is minimized
- (MiniMax) the max path cost over the robots is minimized
- (MinAve) the average time over all target nodes is minimized

(Does not satisfy the task independence assumption discussed in the previous slide.)

We will mainly focus on the MiniSum objective.

Multi-robot Exploration Problems

Several auction-based approaches exist:

- **combinatorial auctions**: compute the TSP for each subset of nodes. Place the bids. Auctioneer computes optimal allocation.

Multi-robot Exploration Problems

Several auction-based approaches exist:

- **combinatorial auctions**: compute the TSP for each subset of nodes. Place the bids. Auctioneer computes optimal allocation.

Problem: not feasible to bid on all bundles!

Multi-robot Exploration Problems

Several auction-based approaches exist:

- **combinatorial auctions**: compute the TSP for each subset of nodes. Place the bids. Auctioneer computes optimal allocation.

Problem: not feasible to bid on all bundles!

- **parallel single-item auctions**: each agent bids on each node separately (smallest path to reach the node) in parallel.

Multi-robot Exploration Problems

Several auction-based approaches exist:

- **combinatorial auctions**: compute the TSP for each subset of nodes. Place the bids. Auctioneer computes optimal allocation.

Problem: not feasible to bid on all bundles!

- **parallel single-item auctions**: each agent bids on each node separately (smallest path to reach the node) in parallel.

Problem: can be arbitrarily far from optimum

Multi-robot Exploration Problems

Several auction-based approaches exist:

- **combinatorial auctions**: compute the TSP for each subset of nodes. Place the bids. Auctioneer computes optimal allocation.

Problem: not feasible to bid on all bundles!

- **parallel single-item auctions**: each agent bids on each node separately (smallest path to reach the node) in parallel.

Problem: can be arbitrarily far from optimum

- **single-item auctions**: each agent bid the cost from their current location to the place they bid for (see next slide)

Single-item Auctions: a first approach

- start with all nodes non allocated
- agents bids on all unallocated nodes (using the **insertion heuristic**: maintaining a path and comparing the cost of inserting the new node in any position in their current path)
- the agent with the winning bid is then temporarily prevented from bidding
- when all agent have the same number of sites to visit, all agents are then allowed to bid again
- in a second phase, agents can implement swap deals to improve the overall allocation

Note: strong constraint to make winning agents unable to bid until all agents have the same number of allocated sites. No guarantee.

Diaz and Stentz. *A free market architecture for distributed control of a multi-robot system*. ICAS-2000.

Parallel Single-item Auctions (PSI)

- Each agent bids on each node independently
- The lowest bid on each node wins that node

Does not take into account any synergy between nodes' locations.
Can be arbitrarily suboptimal.

Single-item Auctions with Guarantees

Can we provide performance guarantees and escape the combinatorial auction computational burden?

Recall that a Minimum Spanning Tree (MST) can be found in polynomial time.

Prim's algorithm: to build the MST, start from a node, then add new nodes by selecting the cheapest edge between any node already in the tree and any node not in the tree.

The MST solution provide a **2-approximation** TSP.

The multi-robot exploration problem seeks a minimum spanning forest (a collection of trees).

Lagoukadis et al. *Simple Auctions with Performance Guarantees for Multi-Robot Task Allocation*. IROS-2004.

Sequential Single-item Auctions (SSI)

SSI are multi-round auctions where one target is allocated at each round:

- each agent estimates the minimum cost from one of its allocated node or the current robot location to each node still available [this is the Prim heuristic version]
- at a given round, each agent submits only its best (=lowest) bid
- the auctioneer collects the bids and assign the node corresponding to the lowest bidding agent

Note: agents only maintain a list of assigned nodes and compute their TSP at the end.

Sequential Single-item Auctions (SSI)

SSI are multi-round auctions where one target is allocated at each round:

- each agent estimates the minimum cost by testing the insertion of the new site in any position in its current path [this is the insertion heuristic version]
- at a given round, each agent submits only its best (=lowest) bid
- the auctioneer collects the bids and assign the target corresponding to the lowest bidding agent

Note: agents maintain a current path which is incremented each time a new node is inserted.

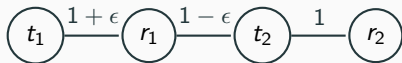
Sequential Single-item Auctions (SSI)

SSI auctions with either Prim or insertion heuristic guarantees a solution which is 2-approximation of the optimal.

Sequential Single-item Auctions (SSI): example

		o_2		o_4			
				r_2			
		r_1			o_1		
				o_3			

SSI: What can go wrong



step 1: r_1 bids $\langle t_2, 1 - \epsilon \rangle$

r_2 bids $\langle t_2, 1 \rangle$

$\Rightarrow r_1$ gets t_2

step 2: r_1 bids $\langle t_1, 2 \rangle$

r_2 bids $\langle t_1, 3 \rangle$

$\Rightarrow r_1$ gets t_1

Overall cost: $(3 - \epsilon)$ while $(2 + \epsilon)$ was optimal.

Intuition: bids on t_2 in the first rounds were very close, so perhaps we could have postponed this allocation without doing any harm.

The Regret-clearing SSI Auction

Koenig et al. propose the following variant of the sequential auction, based on the notion of regret:

- define the **regret of a site** as being the difference between the lowest cost offer of the second lowest cost offer.
- at each turn, the protocol allocates the site with the highest regret to the agent who has made the lowest bid for this site.

Koenig et al. *Agent coordination with regret clearing*. AAAI-2008.

Comparing Properties

Unfortunately, in theory, the regret-clearing SSI version does not offer the same guarantee as the SSI auction. In practice it works very well though.

In terms of **communication**, compare the number of bids induced by each protocol:

- for the basic SSI auction: each agent only bids on **one** site at each round:

$$nm$$

- for the regret-clearing SSI auction, each agent must bid on **all** the non-allocated sites:

$$nm + n(m-1) + n(m-2) + \dots = nm(m+1)/2$$

Consensus-based Auction Algorithms (CBAA)

Auction-based protocols still suffer from a number of limitations:

- these protocols still require that all agents know the locations of all the sites to visit (but not of the other agents) [sometimes called **situational awareness**]
- if one agent plays the role of the auctioneer, then they also introduce a single point of failure. One solution is to let all agents compute broadcast and compute the winning bids themselves. However, not appropriate if communication is not possible between all agents.

Can we address these issues?

Choi et al. *Consensus-Based Decentralized Auctions for Robust Task Allocation*. Robotocs, 2009.

Consensus-based Auction Algorithms (CBAA)

Let us start with the simple matching setting (n nodes to visit, n agents). Assume an undirected communication network G (not necessarily complete, of course).

Each agent i maintains two lists:

- x_i (the binary **task list**, tasks assigned to this agent); and
- y_i (the **winning bid list**, estimate of the highest bid made for each task so far).

The protocol iterates between two phases: the **auctioning** and the **consensus** phase.

Consensus-based Auction Algorithms (CBAA)

- phase 1: auctioning process [similar to Bertsekas, but based on local information only: an unassigned agent computes his bid, based on y_i]
- phase 2: consensus process to converge on the winning bid list
 - agent i send his list of winning bids to all its neighbours in G
 - agent i receives the list of winning bids from all its neighbours in G
 - agent i replaces y_{ij} by the highest value between itself and one received from the neighbours
 - agent i loses a task if it finds out it is outbid for that task

Consensus-based Auction Algorithms (CBAA): example

Run the CBAA algorithm with

	t_1	t_2	t_3	t_4
a_1	5	1	2	3
a_2	1	3	6	1
a_3	1	2	5	3
a_4	8	1	1	1

assuming the (undirected) network

$$G = \{(a_1, a_2), (a_2, a_3), (a_3, a_4), (a_4, a_2)\}$$

Consensus-based Auction Algorithms

Some extensions of this approach:

- extended to the generalized CBAA (CBBA) for the multi-assignment problem.
- because it runs in phases, it still requires some synchronisation among agents (even if bids can be sent asynchronously). The Asynchronous Consensus-Based Bundle Algorithm (ACBBA) addresses this issue.
- recent research suggest that highly dynamic topologies may require different techniques to be efficient (eg. based on Markov Random Fields).

Johnson et al. *Asynchronous Decentralized Task Allocation for Dynamic Environments*. Infotech-Aerospace, 2011.

Pujol-Gonzales et al.. *Decentralized dynamic task allocation for UAVs with limited communication range*. ArXiv-2018.